

# Assignment Assessment Rubric: AI-Proficient Software Developer

---

## Assessment Guidance

This assignment should evaluate both:

- The candidate's ability to build and deliver high-quality software systems.
- The candidate's ability to effectively leverage AI-assisted development while maintaining full engineering ownership and accountability.

Reviewers should assess not only whether the solution works, but also whether the candidate demonstrates:

- Strong software engineering fundamentals
- Sound architectural thinking
- Production-readiness mindset
- Effective use of AI-assisted development
- Critical evaluation of AI-generated outputs
- Engineering judgment and accountability

The candidate should be able to explain:

- Their assumptions and reasoning
- Design and implementation decisions
- Tradeoffs considered
- How AI was used throughout development
- How AI-generated outputs were validated
- Risks, limitations, and future improvements

---

## Scoring Summary

| Category                                      | Weight |
|-----------------------------------------------|--------|
| Problem Understanding & Engineering Reasoning | 15%    |
| Software Design & Architecture                | 20%    |

|                                        |             |
|----------------------------------------|-------------|
| AI-Assisted Development Proficiency    | 20%         |
| Code Quality & Engineering Excellence  | 15%         |
| Testing, Validation & Reliability      | 10%         |
| Security & Production Readiness        | 10%         |
| Operational Excellence & Observability | 5%          |
| Communication & Ownership              | 5%          |
| <b>Total</b>                           | <b>100%</b> |

---

# 1. Problem Understanding & Engineering Reasoning — 15%

Evaluate how well the candidate understands the problem and translates requirements into an engineering solution.

## Look for

- Ability to clarify requirements
- Identification of ambiguities and assumptions
- Structured problem decomposition
- Logical reasoning and decision making
- Consideration of functional and non-functional requirements
- Thoughtful tradeoff analysis
- Awareness of scalability, reliability, security, and maintainability concerns

## Strong Signal

The candidate demonstrates a systematic approach to understanding the problem and makes deliberate engineering decisions supported by clear reasoning.

## Weak Signal

The candidate jumps directly into implementation without showing understanding, analysis, or rationale.

---

## 2. Software Design & Architecture — 20%

Evaluate the quality of the overall technical solution.

### Look for

- Well-defined architecture
- Appropriate use of design patterns
- Clean component boundaries
- API design quality
- Data modeling decisions
- Scalability considerations
- Extensibility and maintainability
- Distributed systems awareness where applicable

### Strong Signal

The solution is architecturally sound, modular, maintainable, and designed with production growth in mind.

### Weak Signal

The design is tightly coupled, difficult to extend, or lacks architectural justification.

---

## 3. AI-Assisted Development Proficiency — 20%

Evaluate how effectively the candidate uses AI as a software engineering accelerator.

### Look for

- Effective use of AI tools throughout the lifecycle
- Structured prompting and task specification
- Problem decomposition before AI usage
- Evidence of iterative refinement

- Ability to distinguish useful from incorrect AI output
- Validation and review of AI-generated code
- Clear accountability for final deliverables
- Understanding of AI limitations and failure modes

### **Strong Signal**

The candidate treats AI as a productivity amplifier rather than a source of truth, actively validating, improving, and governing generated outputs.

### **Weak Signal**

The candidate appears to rely blindly on AI-generated code and cannot explain or defend implementation decisions.

---

## **4. Code Quality & Engineering Excellence — 15%**

Evaluate the quality of the delivered implementation.

### **Look for**

- Readable and maintainable code
- Consistent coding standards
- Appropriate abstractions
- Clean separation of concerns
- Error handling
- Reusability and extensibility
- Performance-conscious implementation
- Documentation quality

### **Strong Signal**

The codebase reflects mature engineering practices and could reasonably be maintained in a production environment.

### **Weak Signal**

Code is difficult to understand, poorly structured, repetitive, or lacks maintainability.

---

## 5. Testing, Validation & Reliability — 10%

Evaluate how rigorously the solution is validated.

### Look for

- Unit testing
- Integration testing
- End-to-end testing where appropriate
- Edge-case coverage
- Failure scenario handling
- Validation of AI-generated outputs
- Test automation mindset
- Clear correctness criteria

### Strong Signal

The candidate demonstrates disciplined validation and can explain how confidence in the solution was established.

### Weak Signal

Minimal testing with only happy-path validation.

---

## 6. Security & Production Readiness — 10%

Evaluate whether the candidate demonstrates production-oriented engineering practices.

### Look for

- Secure coding practices
- Input validation

- Authentication and authorization considerations
- Dependency management awareness
- Secrets management awareness
- Failure containment strategies
- Compliance and governance awareness
- Production deployment considerations

### **Strong Signal**

Security and operational concerns are intentionally designed into the solution from the beginning.

### **Weak Signal**

Security considerations are largely absent or treated as an afterthought.

---

## **7. Operational Excellence & Observability — 5%**

Evaluate the candidate's understanding of operating software in production.

### **Look for**

- Logging strategy
- Monitoring considerations
- Metrics and telemetry
- Error tracking
- Performance measurement
- Reliability considerations
- Incident response awareness

### **Strong Signal**

The candidate demonstrates clear thinking about how the system will be observed, monitored, and maintained.

### **Weak Signal**

No consideration is given to operating the software after deployment.

---

## 8. Communication & Ownership — 5%

Evaluate how effectively the candidate communicates and takes ownership of the solution.

### Look for

- Ability to explain architecture and implementation choices
- Clarity of documentation
- Honest discussion of limitations
- Awareness of tradeoffs
- Ownership of AI-generated artifacts
- Ability to defend engineering decisions
- Professional presentation of the solution

### Strong Signal

The candidate demonstrates accountability, transparency, and deep understanding of every major component.

### Weak Signal

The candidate cannot explain major parts of the solution or attributes decisions solely to AI-generated outputs.

---

## Interview Review Focus

During the discussion, reviewers should ask the candidate to explain:

### Problem Solving

- How they approached the problem
- Key assumptions made
- Alternative solutions considered
- Tradeoffs selected

### Architecture

- System design decisions
- Component boundaries
- Scalability considerations
- Data modeling choices

## AI Usage

- Which AI tools were used
- How prompts were created
- How output quality was assessed
- Examples where AI-generated results were modified or rejected
- How hallucinations or incorrect outputs were detected

## Engineering Discipline

- Testing strategy
- Security considerations
- Reliability concerns
- Code review approach
- Production readiness

## Ownership

- What they would improve with additional time
- Key limitations of the current implementation
- Areas of technical debt
- Lessons learned during development

---

## Scoring Scale

| Score | Meaning                                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Significant gaps in engineering fundamentals and AI proficiency                                                                           |
| 2     | Basic implementation with limited design depth and validation                                                                             |
| 3     | Solid solution demonstrating good engineering practices and effective AI usage                                                            |
| 4     | Strong production-oriented solution with sound architecture and engineering judgment                                                      |
| 5     | Exceptional software engineering execution combined with disciplined, accountable, and highly effective AI-assisted development practices |

---



# Key Assessment Principle

The strongest candidates demonstrate that they can effectively combine strong software engineering fundamentals with AI-assisted development practices while maintaining full accountability for the final solution.

They should exhibit:

- Strong reasoning and problem decomposition
- Sound architectural judgment
- Production-quality implementation practices
- Effective use of AI to accelerate development
- Critical evaluation of AI-generated outputs
- Ownership of all delivered artifacts
- Ability to build, operate, validate, and evolve software systems responsibly in enterprise environments.